Smart Camera &
Video Surveillance System

A Deep Learning based Criminal
Recognition and Tracking System

Week 1 Progress Report

Date: 19 August 2026

Phase: Setup & Face Detection

Status: Completed

Final Year B.Tech (CSE) Major Project

Table of Contents

1. Project Overview
2. System Requirements
3. Installation Steps
4. Project Folder Structure
5. Haar Cascade - How It Works
6. Code Explanation - face_detector.py
7. Code Explanation - webcam_capture.py
8. Code Explanation - image_face_detection.py
9. Code Explanation - dataset_collector.py
10. How to Run the Project
11. Expected Output
12. Week 1 Summary

1. Project Overview

This project builds a Smart Surveillance System that uses Deep Learning and Computer Vision to make CCTV cameras intelligent. Instead of just recording footage, the camera actively watches the video feed, detects faces, and can eventually recognize known individuals.

In Week 1, we have completed the foundation - setting up the development environment, installing all required libraries, and implementing real-time face detection using OpenCV's Haar Cascade classifier. The system can detect faces from a live webcam feed, image files, and video files, drawing bounding boxes around each detected face.

What is Face Detection?

Face detection is the process of locating human faces in images or video frames. It answers the question "Is there a face in this image?" and if yes, "Where is it?" This is different from face recognition, which answers "Whose face is this?" Face detection is the first step in our surveillance pipeline.

2. System Requirements

Hardware Requirements

- Processor: Intel i3 or above (i5 recommended)
- RAM: 4GB minimum (8GB recommended)
- Webcam: Any USB webcam (for live detection)
- Storage: 500MB free space

Software Requirements

- Operating System: Ubuntu 20.04+ / Windows 10+ / macOS
- Python: Version 3.8 or above
- OpenCV: Version 4.8.1 (with contrib modules)
- NumPy: Version 1.24.4
- Haar Cascade XML files (pre-trained models)

3. Installation Steps

Step 1: Check Python Installation

First, verify that Python 3 is installed on your system:

```
python3 --version
# Output: Python 3.8.10
```

```
pip3 --version
# Output: pip 20.0.2
```

Step 2: Create Project Directory

Create a dedicated folder for the project:

```
mkdir -p smart_surveillance/{src,known_faces,snapshots,logs,dataset,models,pdf,config}
```

Step 3: Install OpenCV

Install OpenCV with contrib modules (includes extra face detection algorithms):

```
pip3 install opencv-contrib-python==4.8.1.78
```

Note: We use opencv-contrib-python instead of opencv-python because the contrib version includes additional modules. We also use version 4.8.1 because OpenCV 5.0 removed the CascadeClassifier module which we need.

Step 4: Install NumPy

```
pip3 install numpy
```

NumPy is used for array and numerical operations on image data.

Step 5: Download Haar Cascade Models

Download the pre-trained XML files used for face and eye detection:

```
wget -O models/haarcascade_frontalface_default.xml \
  https://raw.githubusercontent.com/opencv/opencv/master/data/haarcascades/haarcascade_frontalface_defa

wget -O models/haarcascade_eye.xml \
  https://raw.githubusercontent.com/opencv/opencv/master/data/haarcascades/haarcascade_eye.xml
```

4. Project Folder Structure

```
major_project/
|
|-- config/
|   |-- settings.py          # All configuration constants
|
|-- src/
|   |-- face_detector.py     # Core face detection class
|   |-- webcam_capture.py    # Live webcam detection
|   |-- image_face_detection.py  # Image/video detection
|   |-- dataset_collector.py # Collect face dataset
|
|-- models/
|   |-- haarcascade_frontalface_default.xml
|   |-- haarcascade_eye.xml
|
|-- known_faces/             # Store known face images (for later phases)
|-- snapshots/               # Auto-saved detection snapshots
|-- dataset/                 # Collected face datasets
|-- logs/                    # Detection logs
|-- pdf/                     # Project documentation
```

Purpose of Each Folder

- config/: Contains settings.py with all project constants like camera index, email settings, etc.
- src/: Contains all Python source code files
- models/: Contains pre-trained Haar Cascade XML files for face and eye detection
- known_faces/: Will store images of known individuals (for Phase 2)
- snapshots/: Auto-saved screenshots when faces are detected
- dataset/: Collected face images for building the database
- logs/: Detection logs with timestamps
- pdf/: Project documentation

5. Haar Cascade - How It Works

What is Haar Cascade?

Haar Cascade is a machine learning-based approach for object detection, originally proposed by Paul Viola and Michael Jones in 2001. It uses a series of simple binary features (called Haar-like features) to detect objects in images. For face detection, it was trained on thousands of face and non-face images.

How It Detects Faces

The algorithm works in four main stages:

Stage 1: Haar-Like Feature Extraction

The classifier looks at rectangular regions in the image and calculates the difference in intensity between adjacent rectangles. For example, the eye region is typically darker than the forehead, so the feature detector identifies this pattern.

Stage 2: Integral Image

To compute these features quickly, the algorithm uses an "integral image" which allows summing pixel values in any rectangular region in constant time (O(1)).

Stage 3: AdaBoost Training

AdaBoost selects the most important features from a large pool and combines them into a strong classifier. Each weak classifier looks at one feature and makes a simple decision.

Stage 4: Cascade Architecture

Classifiers are arranged in a cascade (series of stages). Simple stages reject non-faces early. Only regions that pass all stages are classified as faces. This makes the algorithm very fast because most image regions are rejected early.

Key Parameters in detectMultiScale()

The detectMultiScale() function has several important parameters:

Parameter	Value	Description
scaleFactor	1.3	How much the image size is reduced at each image scale
minNeighbors	5	How many neighbors each candidate rectangle should have
minSize	(50,50)	Minimum possible object size. Objects smaller are ignored

scaleFactor=1.3 means the image is scaled down by 30% at each step. This helps detect faces of different sizes. A smaller value (like 1.1) is more accurate but slower.

minNeighbors=5 means a detection must have at least 5 overlapping detections to be considered a face. Higher values mean fewer false positives but might miss some faces.

6. Code Explanation - face_detector.py

This is the core module that handles all face detection logic. It is designed as a reusable class that can be imported by other modules.

Class: FaceDetector

The FaceDetector class has four main methods:

__init__(self)

Constructor method. Loads the Haar Cascade XML files for face and eye detection. Raises an error if the files cannot be loaded.

detect_faces(self, frame)

Takes a video frame (as a NumPy array) and returns a list of dictionaries, each containing the face position (x, y, width, height), number of eyes detected, and timestamp.

draw_detections(self, frame, detections)

Draws green bounding boxes and labels on the frame for each detected face. Returns the annotated frame for display.

save_snapshot(self, frame, detection)

Saves a cropped image of the detected face with some padding. Files are saved with timestamp-based filenames to avoid conflicts.

7. Code Explanation - webcam_capture.py

This module handles live webcam capture and real-time face detection. It creates a continuous loop that reads frames from the webcam, passes them to the FaceDetector, and displays the results.

Flow of Execution

- Step 1: Initialize FaceDetector and open webcam (camera index 0)
- Step 2: Set resolution to 640x480
- Step 3: Enter infinite loop - read frame, detect faces, draw boxes, display
- Step 4: Show info bar with face count, total detections, and elapsed time
- Step 5: Handle keyboard inputs - q to quit, s to snapshot, r to reset counter

Keyboard Controls

Key	Action
q	Quit the application
s	Save a snapshot of detected faces
r	Reset the detection counter

8. Code Explanation - image_face_detection.py

This module provides two functions for detecting faces in static images and pre-recorded video files.

detect_faces_in_image(image_path)

Reads an image from the given path, detects all faces, draws bounding boxes, saves the output image to the snapshots folder, and displays it in a window.

detect_faces_in_video(video_path)

Opens a video file, processes every 3rd frame (to improve speed), detects faces, and displays results in real-time. Press q to stop processing early.

9. Code Explanation - dataset_collector.py

This module collects face images from the webcam to build a dataset of known individuals. This dataset will be used in Phase 2 for face recognition (matching faces against a database).

How It Works

- Takes a person name as argument (e.g., "john")
- Opens webcam and detects the face
- Press "c" to capture a frame - face is cropped, resized to 200x200, and saved
- Default: collects 50 samples per person
- Images are saved in dataset/<person_name>/ folder

Usage

```
python3 dataset_collector.py john
python3 dataset_collector.py john 100  # collect 100 samples
```

10. How to Run the Project

Option A: Live Webcam Detection

```
cd major_project
python3 -m src.webcam_capture
```

This opens your webcam and starts detecting faces in real-time.

Option B: Detect Faces in an Image

```
python3 -m src.image_face_detection path/to/image.jpg
```

This detects faces in a static image and saves the result.

Option C: Detect Faces in a Video

```
python3 -m src.image_face_detection --video path/to/video.mp4
```

This processes a video file and shows face detections frame by frame.

Option D: Collect Face Dataset

```
python3 -m src.dataset_collector person_name [num_samples]
```

This captures face images from webcam to build the database.

11. Expected Output

Webcam Detection Output

When you run webcam_capture.py, a window opens showing your webcam feed. Each detected face gets a green bounding box around it with a label showing the number of eyes detected. An info bar at the top shows:

- Number of faces currently visible
- Total number of faces detected since start
- Elapsed time since start

Image Detection Output

When you run image_face_detection.py with an image, it displays the image with green bounding boxes around all detected faces. The console prints the position and size of each face found.

Snapshot Saving

Press "s" during webcam detection to save a snapshot. The cropped face image is saved in the snapshots/ folder with a timestamp filename like snapshot_20260819_143022.jpg.

12. Week 1 Summary

What Was Accomplished

- Set up the complete project directory structure
- Installed OpenCV 4.8.1 with contrib modules
- Installed NumPy 1.24.4
- Downloaded pre-trained Haar Cascade XML models
- Created modular Python codebase with 4 source files
- Implemented real-time face detection from webcam
- Implemented face detection on images and videos
- Created dataset collection tool for future phases
- All modules tested and verified working

Technologies Used This Week

Technology	Purpose
Python 3.8.10	Main programming language
OpenCV 4.8.1	Image processing and face detection
NumPy 1.24.4	Array operations for image data
Haar Cascade	Pre-trained face/eye detection model

What Will Be Done in Week 2

- Install dlib and face_recognition libraries
- Implement face encoding (converting faces to numerical vectors)
- Build the face database using collected dataset
- Implement face matching/comparison logic
- Test recognition accuracy with known individuals

Issues Faced

- OpenCV 5.0 removed CascadeClassifier - resolved by using opencv-contrib-python 4.8.1
- dlib compilation from source takes very long - installation running in background